Software Configuration Management

Software Configuration Management

- **Software Configuration Management (SCM)** is a systematic process used to manage and control changes to software products during their development and maintenance.
- It ensures consistency, traceability, and integrity of software artifacts like source code, documentation, and test scripts.

Configuration Item

- A configuration item (CI) is any service component, infrastructure element, or other item that needs to be managed in order to ensure the successful delivery of services.

Examples of CI types are:

- Hardware/devices
- Software/applications
- Communications/networks
- System
- Location
- Facility
- Database
- Service

Configuration Management Activities

- **Configuration management (CM) activities** ensure that all software and system artifacts are systematically managed, tracked, and maintained throughout the development lifecycle.
- These activities are designed to handle changes, maintain consistency, and ensure the integrity of the software product.

1. Change Management
2. Version Management
3. System Building
4. System Release

Configuration Management

- New versions of software systems are created as they change:
  - For different machines/OS;
  - Offering different functionality;
  - Tailored for particular user requirements.
- Configuration management is concerned with managing evolving software systems:
  - System change is a team activity;
  - CM aims to control the costs and effort involved in making changes to a system.

Configuration Management(CM)

- Involves the development and application of procedures and standards to manage an evolving software product.
- CM may be seen as part of a more general quality management process.
- When released to CM, software systems are sometimes called baselines as they are a starting point for further development.

Configuration Management Planning

- All products of the software process may have to be managed:
  - Specifications
  - Designs
  - Programs
  - Test data
  - User manuals.
- Thousands of separate documents may be generated for a large, complex software system.

The CM Plan

- Defines the types of documents to be managed and a document naming scheme.
- Defines who takes responsibility for the CM procedures and creation of baselines.
- Defines policies for change control and version management.
- Defines the CM records which must be maintained.
- Describes the tools which should be used to assist the CM process and any limitations on their use.
- Defines the process of tool use.
- Defines the CM database used to record configuration information.
- May include information such as the CM of external software, process auditing, etc.

The Configuration Database

- All CM information should be maintained in a configuration database.
- This should allow queries about configurations to be answered:
  - Who has a particular system version?
  - What platform is required for a particular version?
  - What versions are affected by a change to component X?
  - How many reported faults in version T?
- The CM database should preferably be linked to the software being managed.

CM Database Implementation

- May be part of an integrated environment to support software development.
  - The CM database and the managed documents are all maintained on the same system
- CASE tools may be integrated with this so that there is a close relationship between the CASE tools and the CM tools.
- More commonly, the CM database is maintained separately as this is cheaper and more flexible.

CASE Tools For Configuration Management

- CM processes are standardised and involve applying pre-defined procedures.
- Large amounts of data must be managed.
- CASE tool support for CM is therefore essential.
- Mature CASE tools to support configuration management are available ranging from stand-alone tools to integrated CM workbenches.

Change Management

- Software systems are subject to continual change requests:
  - From users;
  - From developers;
  - From market forces.
- Change management is concerned with keeping track of these changes and ensuring that they are implemented in the most cost-effective way.

The Change Management Process

Request change by completing a change request form
Analyze change request
if change is valid **then**
    Assess how change might be implemented
    Assess change cost
    Submit request to change control board
    if change is accepted **then**
        repeat
            make changes to software
            submit changed software for quality approval
        until software quality is adequate
        create new system version
    else
        reject change request
else
    reject change request

Change Request Form

- The definition of a change request form is part of the CM planning process.
- This form records the change proposed, requestor of change, the reason why change was suggested and the urgency of change(from requestor of the change).
- It also records change evaluation, impact analysis, change cost and recommendations (System maintenance staff).

Change Request Form

Change Request Form

Project: Proteus/PCL-Tools **Number:** 23/02
Change requester: I. Sommerville **Date:** 1/12/02
Requested change: When a component is selected from the structure, display the name of the file where it is stored.

Change analyser: G. Dean **Analysis date:** 10/12/02
Components affected: Display-Icon.Select, Display-Icon.Display

Associated components: FileTable

Change assessment: Relatively simple to implement as a file name table is available. Requires the design and implementation of a display field. No changes to associated components are required.

Change priority: Low

Change implementation:
Estimated effort: 0.5 days

Date to CCB: 15/12/02 **CCB decision date:** 1/2/03
CCB decision: Accept change. Change to be implemented in Release 2.1.
Change implementor: **Date of change:**
Date submitted to QA: **QA decision:**
Date submitted to CM:
Comments

Change Control Board(CCB)

- Changes should be reviewed by an external group who decide whether or not they are cost-effective from a strategic and organizational viewpoint rather than a technical viewpoint.
- Should be independent of project responsible for system.
- The group is sometimes called a change control board.
- The CCB may include representatives from client and contractor staff.

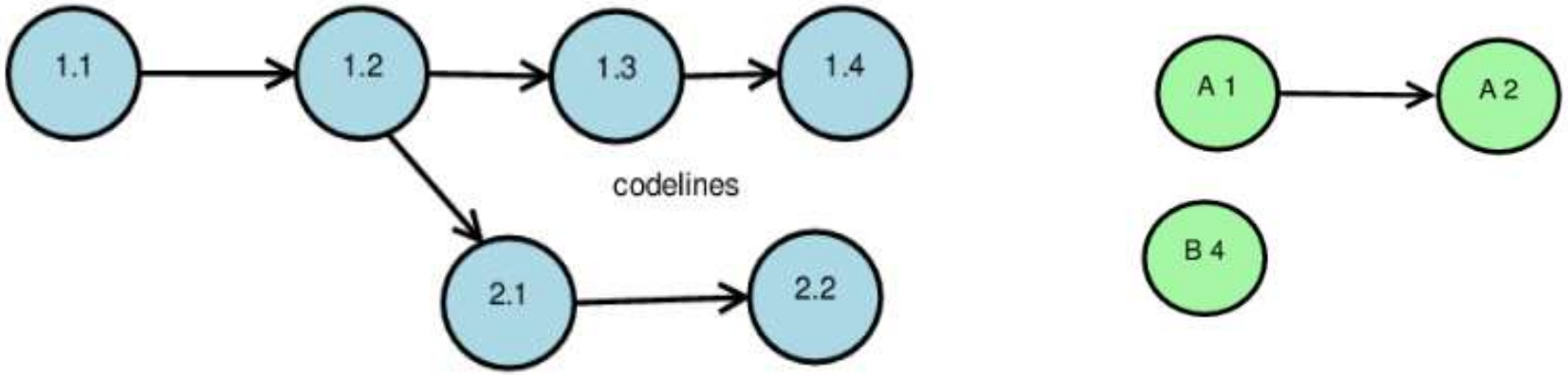
Version Management

- Version Management also called Version Control or Revision Control, is a means to effectively track and control changes to a collection of related entities.
- Version control, also known as source control, is the practice of tracking and managing changes to software code.
- Version control systems are software tools that help software teams manage changes to source code over time.
- Version control software keeps track of every modification to the code in a special kind of database.
- If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.

Version Management

Codelines

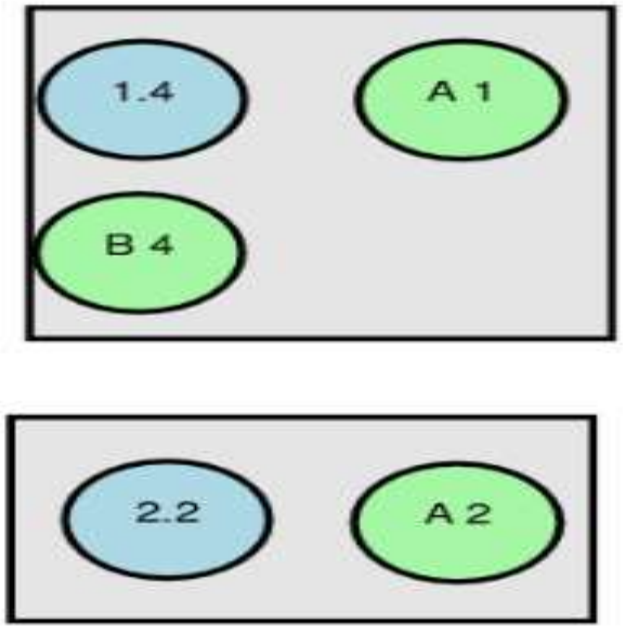
- A **codeline** refers to a specific sequence of versions of source code files that evolves over time as changes are made.
- Codelines are foundational to managing software development in projects where multiple versions or variations of the software are being developed simultaneously.



Version Management

Baselines

- A **baseline** refers to a stable and well-defined reference point in the software development lifecycle.
- It represents a version or set of configuration items (e.g., code, documents, test plans) that has been formally reviewed and approved.
- Baselines are used as a foundation for further development, ensuring that changes are controlled and traceable.



Key Benefits of Version Control

1. Organized, coordinated management of changes to software assets by one or many individuals, some of whom may be geographically dispersed.
2. Organized, coordinated management of changes to software assets for emergency hot-fixes, routine maintenance, upgrades, and new features with potentially overlapping development timeframes.
3. An auditable change history (e.g., what changed, when, and by whom)
4. A reliable master copy of what assets are currently in production
5. A reliable master copy of assets from which to build and/or configure the production environment
6. Reliable copies of previous production versions of assets
7. Ability to see the specific differences between distinct versions of a given asset

Release Management

- Release Management is the process responsible for planning, scheduling, and controlling the build, in addition to testing and deploying Releases.
- Release management oversees all the stages involved in a software release from development and testing to deployment.
- Release management is required anytime a new product or even changes to an existing product are requested.
- Steps to release management:

1. Plan release
2. Build release
3. User acceptance testing
4. Prepare release
5. Deploy release